

Chapitre 3 : Mesurer la complexité

Bruno LEGRIX

15/11/2024

1 Introduction

En mathématiques, la comparaison asymptotique est une méthode consistant à étudier la vitesse de croissance d'une fonction au voisinage d'un point ou à l'infini, en la comparant à celle d'une autre fonction considérée comme plus "simple". Celle-ci est souvent choisie sur une échelle de référence, contenant en général les polynômes, les exponentielles et les logarithmes. Les notations correspondantes sont en particulier utilisées en physique et en informatique, par exemple pour décrire la complexité de certains algorithmes.

Dans ce cours, nous n'allons étudier qu'au voisinage de l'infini, car nous n'utiliserons que des fonctions à variable entière n , donc on ne précisera pas que n tend vers $+\infty$.

2 Domination

La notation grand O de Landau (1909) dénote le caractère dominé d'une fonction par rapport à une autre. Ce symbole a été introduit primitivement en 1894 par Paul Bachmann, on utilise la lettre O majuscule (de l'allemand Ordnung, "ordre").

Soient f et g deux fonctions

On dit que f est dominée par g , ou que g domine f , et on note :

$$f(n) = O(g(n))$$

lorsqu'il existe des constantes N et C telles que :

$$\forall n > N \quad |f(n)| \leq C|g(n)|$$

Intuitivement, cela signifie que f ne croît pas plus vite que g .

Une autre caractérisation plus commode dans le cas où g ne s'annule pas à partir d'un certain point :

$$f(n) = O(g(n)) \text{ si et seulement si } \frac{f(n)}{g(n)} \text{ est bornée}$$

La notation grand O "ignore" les constantes multiplicatives, donc on a $O(\lambda \times g(n)) = O(g(n))$.

Par exemple, on a $O(\log_2(n)) = O(\log_{10}(n))$.

La notation grand O est la notation standard utilisée pour décrire la complexité des algorithmes. Cette notation, le cas de la complexité temporelle, indique la croissance asymptotique du temps d'exécution en fonction de la taille de l'entrée.

Propriétés

- 1) La domination est une relation de comparaison.
- 2) Elle est transitive, mais n'est ni réflexive, ni symétrique.
- 3) Si $f_1 = O(g)$ et $f_2 = O(g)$ alors $f_1 + f_2 = O(g)$
On note $O(g) + O(g) = O(g)$
- 4) Si $f_1 = O(g_1)$ et $f_2 = O(g_2)$ alors $f_1 f_2 = O(g_1 g_2)$

Cette notation s'appelle la limite supérieure de l'algorithme ou le pire des cas d'un algorithme. Cela nous indique qu'une certaine fonction ne dépassera jamais une durée spécifiée pour une valeur quelconque de l'entrée n .

3 Négligeabilité (ou prépondérance)

La notation petit o de Landau (1909) dénote le caractère négligeable d'une fonction par rapport à une autre.

Soient f et g deux fonctions.

On dit que f est négligeable devant g , ou que g prépondérante devant f , et on note :

$$f(n) = o(g(n))$$

lorsqu'il existe une constante N et une fonction ϵ qui tend vers 0 telles que :

$$\forall x > N \quad f(n) = \epsilon(n) \times g(n)$$

Intuitivement, cela signifie que f devient insignifiante devant g .

Une autre caractérisation plus commode dans le cas où g ne s'annule pas à partir d'un certain point :

$$f(n) = o(g(n)) \text{ si et seulement si } \frac{f(n)}{g(n)} \rightarrow 0$$

Propriétés

- 1) La négligeabilité est une relation de comparaison.
- 2) Elle est transitive, mais n'est ni réflexive, ni symétrique.
- 3) Si $f_1 = o(g)$ et $f_2 = o(g)$ alors $f_1 + f_2 = o(g)$
On note $o(g) + o(g) = o(g)$
- 4) Si $f_1 = o(g_1)$ et $f_2 = o(g_2)$ alors $f_1 f_2 = o(g_1 g_2)$
- 5) Si $f = o(g)$ alors $f + g = O(g)$

Ainsi, on a utilisé les négligences suivantes :

- $1 = o(\log(n))$
- $\log(n) = o(n)$
- $n = o(n \log(n))$ qui est la conséquence de la 4^{ème} propriété
- $n \log(n) = o(n^2)$, d'où $n = o(n^2)$ (transitivité)
- $n^2 = o(2^n)$

4 La notation Ω (grand omega)

4.1 En théorie analytique des nombres

En 1914, Hardy et Littlewood ont introduit le nouveau symbole Ω défini ainsi :

$$f(n) = \Omega(g(n)) \Leftrightarrow \limsup \left| \frac{f(n)}{g(n)} \right| > 0$$

Ainsi $f(n) = \Omega(g(n))$ est la négation de $f(n) = o(g(x))$.

4.2 Deux définitions incompatibles

Il est important de souligner le fait que l'écriture $f(n) = \Omega(g(n))$ possède en mathématiques deux définitions incompatibles, toutes les deux très répandues, l'une en théorie analytique des nombres, qu'on vient de présenter, l'autre en théorie de la complexité des algorithmes. Lorsque ces deux disciplines se rencontrent, cette situation est susceptible de créer une grande confusion.

4.3 Dans la théorie de la complexité des algorithmes

En 1976, Knuth publie un article dont le but principal est de justifier son utilisation du même symbole Ω pour décrire une autre propriété que celle décrite par Hardy et Littlewood. Il tente de convaincre le lecteur que la définition de Hardy et Littlewood n'est quasiment jamais utilisée (ce qui est faux). Il prend donc le risque de créer la confusion, et définit

$$f(n) = \Omega(g(n)) \Leftrightarrow g(n) = O(f(n))$$

La notation Ω est utilisée pour définir la limite inférieure de tout algorithme ou nous pouvons dire le meilleur des cas de tout algorithme.

Ceci indique toujours le temps minimum requis pour tout algorithme pour toutes les valeurs d'entrée, donc le meilleur des cas pour tout algorithme.

En termes simples, lorsque nous représentons une complexité temporelle pour tout algorithme sous la forme de Ω , nous voulons dire que l'algorithme prendra au moins ce temps pour terminer son exécution. Cela peut certainement prendre plus de temps que cela aussi.

5 De l'ordre de ; Θ (grand theta)

$f(n) = \Theta(g(n))$ quand il existe des constantes c_1 et c_2 telles qu'à partir d'un certain point, on ait :

$$c_1|g(n)| \leq |f(n)| \leq c_2|g(n)|$$

On dit alors que les fonctions f et g sont du même ordre de grandeur asymptotiquement.

Cela signifie que la fonction f est bornée par la fonction g .

Un moyen simple d'obtenir la notation Thêta d'une expression consiste à supprimer les termes d'ordre faible et à ignorer les constantes principales.

Nous entendons que la complexité temporelle représentée par la notation Θ est similaire à la valeur moyenne ou à la plage dans laquelle le temps réel d'exécution de l'algorithme sera.

La question est de savoir pourquoi nous avons besoin de O alors que nous avons déjà la notation Θ , qui représente le temps d'exécution étroitement lié à tout algorithme.

Prenons un petit exemple pour comprendre cela. Considérons un algorithme de recherche linéaire, dans lequel nous parcourons les éléments du tableau, un par un, pour rechercher un nombre donné. Dans le pire des cas, en partant du début du tableau, nous trouvons l'élément ou le nombre que nous recherchons à la fin, ce qui conduit à une complexité temporelle de n . Mais il peut arriver que l'élément recherché soit le premier élément du tableau, auquel cas la complexité temporelle sera de 1.

Maintenant, dans ce cas, dire que la recherche linéaire est de complexité $\Theta(n)$ signifie que le temps requis sera toujours lié à n , car c'est le bon moyen de représenter la complexité moyenne du temps, mais lorsque nous utilisons la notation O , nous entendons par là que la complexité temporelle est $O(n)$, ce qui signifie que la complexité temporelle ne dépassera jamais n , définissant ainsi la limite supérieure, ce qui signifie qu'elle peut être inférieure à ou égal à n , qui est la représentation correcte. C'est la raison pour laquelle, la plupart du temps, vous verrez que la notation O est utilisée pour représenter la complexité temporelle d'un algorithme, car cela a plus de sens.

6 Règles

On suppose qu'on ait :

- `instruction1` de complexité temporelle en $\Theta(f_1(n))$
- `instruction2` de complexité temporelle en $\Theta(f_2(n))$
- L'évaluation de la condition `condition` est en $\Theta(f(n))$

6.1 Règle 1 : Composition Séquentielle

Le bloc d'instructions :

```
instruction1 ;
instruction2
```

a une complexité temporelle en $\Theta(\max(f_1(n), f_2(n)))$

6.2 Règle 2 : `if (condition) instruction1 else instruction2`

Alors la complexité de l'instruction

```
if (C) instruction1 else instruction2
```

est en $O(\max(f(n), f_1(n), f_2(n)))$

6.3 Règle 3 : for

On suppose que `instruction1` n'a aucun effet sur les variables `i` et `n`, la complexité temporelle de la boucle

```
for (int i=0 ; i < n ; i++)  
    instruction1
```

est en $\Theta(n * f_1(n))$

Si `instruction1` se trouve au cœur de `k` boucles `for` imbriquées, chacune d'elle de la forme

```
for (int im = 0 ; im < n ; im++)
```

où $0 < m < (k+1)$ combien de fois `instruction1` est elle exécutée ?

6.4 Règle 4 : while (condition) instruction1

La boucle `while` est exécutée en $\Theta(g(n))$ fois

```
while (condition)    instruction1
```

est en $\Theta(g(n) * \max(f(n), f_1(n)))$

Exercice n° 1

Estimer les complexités des morceaux de codes suivants, sachant que `instruction1` est en $\Theta(1)$ et `instruction1` ne modifie pas les entiers `i`, `j`, `k` et `n`

```
for (int i=0 ; i<n ; i++) {  
    for (int j=i ; j<n ; j++) {  
        for (int k=0 ; k<j ; k++)  
            instruction1
```

Exercice n° 2

Estimer les complexités des morceaux de codes suivants, sachant que `instruction1`, `instruction2` et `instruction3` sont en $\Theta(1)$ et ne modifient pas les entiers `i`, `j`, `k` et `n`

```
int i=1 ;  
int j=1 ;  
while (i < n) {  
    i++ ;  
    instruction1 ;  
    while ((j < n) && condition) {  
        j++ ;  
        instruction2  
    }  
    instruction3 ;  
}
```

6.5 Règle 5 : Composition de Méthodes

Nous avons :

- `methode1(Classe1 o1)` en $O(f_1(taille1(o1)))$
- `methode2(Classe2 o2) : Classe1` en $O(f_2(taille2(o2)))$

La complexité de `methode1(methode2(o2))` est en $O(\max(f_2(taille2(o2)), f_1(taille1(methode2(o2))))$

Exercice n° 3

On connaît l'écriture d'un nombre en base b , et l'on veut convertir ce nombre en base usuelle (base dix).

- 1) On utilise la méthode `conversionDirecte`. Quelle en est la complexité ?

```
public int conversionDirecte(int[] a, int b) {
    int resultat = a[0] ;
    int auxiliaire ;
    for (int rang = 1 ; rang < a.length ; rang++) {
        if (a[rang] != 0) {
            auxiliaire = a[rang] ;
            for (int indice = 0 ; indice < rang ; indice++) {
                auxiliaire = auxiliaire * b ;
            }
            resultat = resultat + auxiliaire;
        }
    }
    return resultat ;
}
```

- 2) On utilise la méthode `conversionDirecteV2` dans laquelle on mémorise dans une variable monome b^{rang} . Ecrire cette méthode `conversionDirecteV2` et en donner la complexité ?
- 3) Prouvez que la méthode suivante dite de Horner, effectue bien le même travail. Quelle en est la complexité ?

```
public int horner(int[] a, int b) {
    int n = a.length ;
    int resultat = a[n-1] ;
    for (int rang = n-2 ; rang >= 0 ; rang--){
        resultat = b * resultat + a[rang] ;
    }
    return resultat ;
}
```

7 Le théorème sur les récurrences de partition

Le théorème sur les récurrences de partition, ou “théorème général”, ou “master theorem” (1980), permet d’obtenir une solution en termes asymptotiques (en utilisant les notations en O) pour des relations de récurrence d’un certain type rencontrées dans l’analyse de complexité d’algorithmes qui sont régis par le paradigme diviser pour régner.

Considérons un problème qui peut être résolu par un algorithme récursif de la famille diviser pour régner qui opère comme suit : on partage une instance de taille n en sous-problèmes. Il y a a sous-problèmes et chacun est de taille n/b . On résout ces sous-problèmes puis on recombine les solutions partielles en une solution du problème initial. On peut imaginer cette approche comme la construction d’un arbre, où chaque nœud est un appel récursif, et les enfants sont les instances appelées. Dans le cas décrit ci-dessus, chaque nœud possède a enfants. Chaque nœud répartit les données entre ses enfants et récupère les résultats partiels pour les synthétiser et obtenir la solution globale. La répartition des données entre enfants et l’intégration des résultats ont bien entendu également un coût qui, pour un problème de taille n , est noté par $f(n)$.

La complexité en temps des algorithmes de cette nature est exprimée par une relation de récurrence de type :

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

On peut développer cette relation, en substituant la définition dans la partie droite de l’équation pour obtenir une expression pour le coût total au cas par cas. Mais une expression comme celle fournie par le “master theorem”, permet d’exprimer la complexité asymptotique sans passer par un développement explicite.

On peut déterminer des bornes asymptotiques exactes dans trois cas, selon l’importance du surcoût mesuré par la croissance de la fonction $f(n)$. Pour cela, on exprime sa croissance par l’ordre de grandeur n^c , où c mesure le rapport entre a et b^c . Donc on compare c et $\log_b a$.

Les trois cas couverts par le théorème sont :

- 1) $f(n) = O(n^c)$ où $c < \log_b a$
 Dans ce cas, on a : $T(n) = \Theta(n^{\log_b a})$
- 2) $f(n) = \Theta(n^c \log^k n)$ où k est une constante et $c = \log_b a$
 Dans ce cas, on a : $T(n) = \Theta(n^c \log^{k+1} n)$
- 3) $f(n) = \Omega(n^c)$ où $c > \log_b a$ et de plus $a.f(n/b) \leq kf(n)$ (cette condition est appelée parfois la « condition de régularité ») pour une constante $k < 1$ et n assez grand.
 Dans ce cas, on a : $T(n) = \Theta(f(n))$

Supposons la relation de récurrence suivante :

$$T(n) = a T\left(\frac{n}{b}\right) + O(n^d)$$

Le “théorème général” permet d’obtenir une expression de la complexité de $T(n)$ comme suit :

- 1) Si $d < \log_b a$ alors $T(n) = O(n^{\log_b a})$.
- 2) Si $d = \log_b a$ alors $T(n) = O(n^d \log_b n)$.
- 3) Si $d > \log_b a$ alors $T(n) = O(n^d)$.

Exercice n° 4

Donner la relation de récurrence, c'est-à-dire déterminer a , b et $f(n)$ dans $T(n) = aT(\frac{n}{b}) + f(n)$. comment appliquer le "théorème général" ? Et en déduire la complexité en temps.

- 1) Recherche dichotomique
- 2) Parcours d'arbre binaire
- 3) Trouver la valeur médiane
- 4) Tri fusion

Exercice n° 5

Aux équations suivantes dans les recherches de complexité d'algorithmes récursifs, donner la solution :

- 1) $T(n) = T(n-1)$
- 2) $T(n) = T(n-1) + 1$
- 3) $a > 1, T(n) = aT(n-1) + n^k$
- 4) $T(n) = 2T(n/2) + 1$
- 5) $T(n) = 2T(n/2) + n$
- 6) $T(n) = 2T(n/2) + n^k$

Exercice n° 6 Tour de Hanoi

```
public class TourDeHanoi {
    public static void main(String[] args) {
        tourDeHanoi(7, 'A', 'B', 'C');
    }
    public static void tourDeHanoi(int n, char depart, char inter, char arrivee) {
        if (n==1) {
            System.out.println("disque "+1+" depart "+depart+" arrivee "+arrivee);
        } else {
            tourDeHanoi(n-1, depart, arrivee, inter);
            System.out.println("disque "+n+" depart "+depart+" arrivee "+arrivee);
            tourDeHanoi(n-1, inter, depart, arrivee);
        }
    }
}
```

Calculer la complexité de `tourDeHanoi`

Exercice n° 7 Nombres De Fibonacci

Leonardo de Pise, surnommé Fibonacci est un mystère de l'histoire des mathématiques. Il serait né vers 1175 et mort en 1240, et aurait vécu toute sa vie à Pise. Il a publié un unique livre, Liber Abaci (une œuvre collective?).

Reproduction des lapins : "Possédant au départ un couple de lapins, combien de couples de lapins obtient-on en douze mois si chaque couple engendre tous les mois un nouveau couple à compter du second mois de son existence?"

On note $F(n)$ le nombre de couples de lapins au mois n . Donc

$F(n)$ = nombre de couples au mois $(n - 1)$ + nombre de couples nés au mois n

D'où $F(n) = F(n - 1) + F(n - 2)$

- 1) Calculer la complexité de l'implémentation suivante :

```
public int fibonacci (int n) {
    if (n==0) return 0 ;
    if (n==1) return 1 ;
    return fibonacci(n-1)+fibonacci(n-2);
}
```

- 2) Calculer la complexité de l'implémentation suivante :

```
public int fibonacci(int n ){
    int i = 2;
    int fiMoins2 = 0 ; // f(0) = 0
    int fiMoins1 = 1 ; // f(1) = 1
    int fi = 1 ;      // f(2) = 1
    for (int i = 3 ; i < n+1 ; i++) {
        // mise à jour de fiMoins2 et de fiMoins1
        fiMoins2 = fiMoins1;
        fiMoins1 = fi;
        // calcul de fi qui est égal au ième terme de la suite
        fi= fiMoins2 + fiMoins1;
    }
    // fi est le n ième terme de la suite pour tout n > 0
    if (n==0) return 0 ;
    else return fi ;
}
```

- 3) Les mathématiciens savent que :

$$F(2k) = (2F(k+1) - F(k))F(k) \text{ et } F(2k+1) = F(k)^2 + F(k+1)^2$$

Calculer la complexité de l'implémentation suivante :

```
public int[] fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacciRecursif(n)[0];
}
```

```

public int[] fibonacciRecuratif(int n) {
    if (n == 0) return new long[]{0, 1};

    int[] tmp = fibonacciRecuratif(n/2);
    int  a    = (2*tmp[1]-tmp[0])*tmp[0];
    int  b    = tmp[0]*tmp[0] + tmp[1]*tmp[1];

    if (n%2 == 0) return new int[]{a, b};
    else          return new int[]{b, a+b};
}

```

- 4) Calculer la complexité de l'implémentation suivante :

```

public static int[] decompose(int n) {
    int p = Integer.toBinaryString(n).length();
    int[] decomposition = new int[p];
    for (int indice = 0 ; indice < p ; indice++) {
        decomposition[indice] = n % 2 ;
        n = n / 2;
    }
    return decomposition;
}

public static int fibonacciDecompose (int n) {
    int a = 0;
    int b = 1;
    int tmp ;
    int[] decomposition = decompose(n) ;
    int p = decomposition.length;
    for (int indice = 1 ; indice <= p ; indice++) {
        tmp = a;
        a   = (2*b-a)*a;
        b   = tmp*tmp + b*b;
        if (decomposition[p-indice]==1) {
            b = a+b;
            a = b-a;
        }
    }
    if (n == 1) return 1;
    else return a ;
}

```